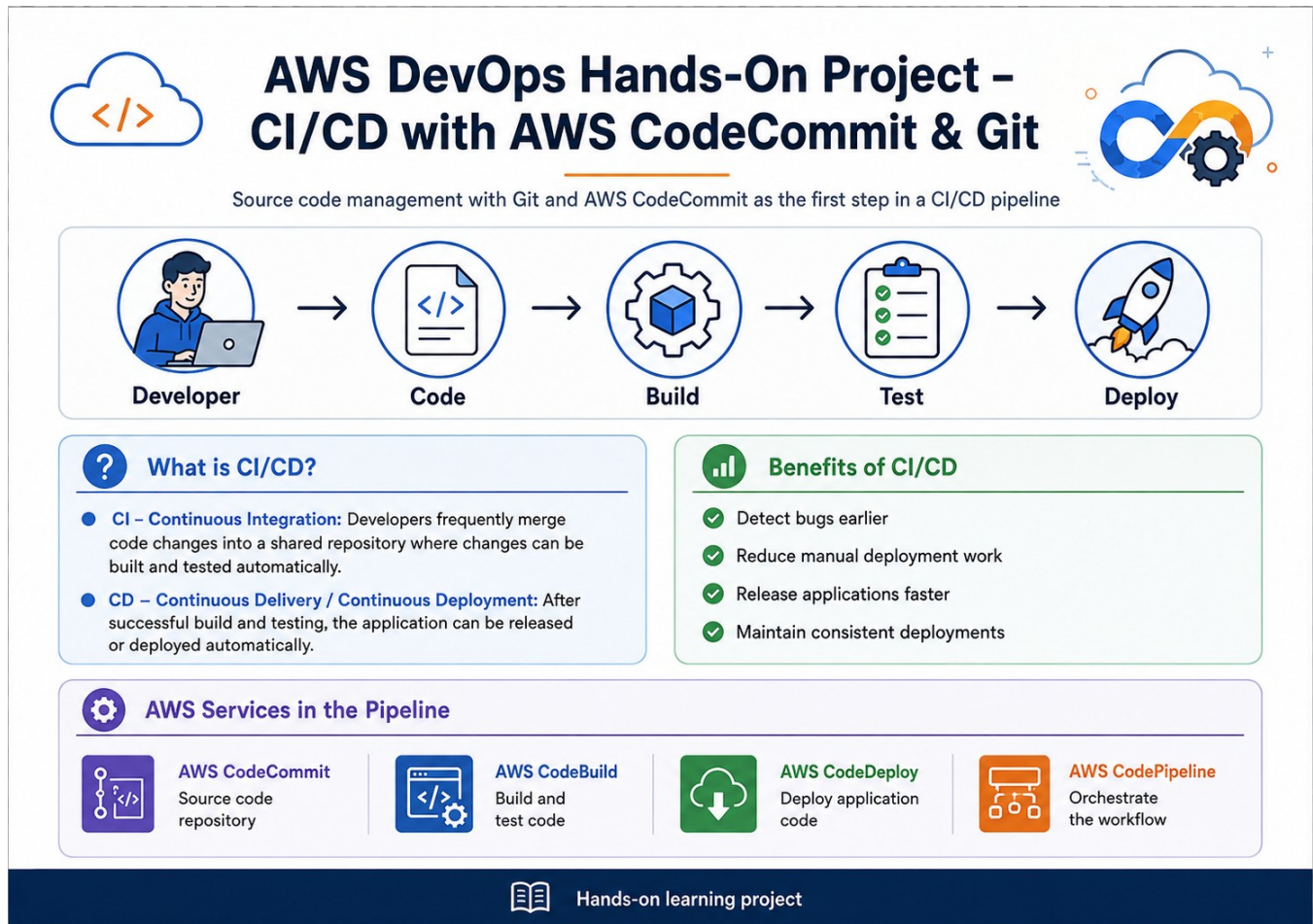


AWS DEVOPS HANDS-ON PROJECT

CI/CD with AWS CodeCommit & Git

Source Code Version Control • Git Workflow • AWS DevOps Fundamentals



Project: codecommit_demo

Prepared as GitHub project documentation

1. Project Overview

This hands-on project demonstrates how application source code moves from a local Windows development environment into an AWS CodeCommit Git repository. The focus is on source control, authentication, Git operations, and the role of CodeCommit as the source stage of a CI/CD workflow.

Scope completed: IAM setup → CodeCommit repository → Git credential configuration → clone → create code → add → commit → push → browser verification.

Project Objectives

- Understand the purpose of Continuous Integration and Continuous Delivery/Deployment.
- Create and securely access a Git-based source code repository in AWS CodeCommit.
- Practice the standard Git workflow from a Windows workstation.
- Understand how source control becomes the starting point of a complete AWS CI/CD pipeline.
- Prepare the repository for future integration with CodeBuild, CodeDeploy, and CodePipeline.

Project Flow

Developer	Git	CodeCommit	Build/Test	Deploy
-----------	-----	------------	------------	--------

*Current hands-on implementation stops at the CodeCommit source stage.
Build, test, and deployment automation are the next phase.*

2. CI/CD Fundamentals

Continuous Integration (CI)

CI is the practice of frequently integrating code changes into a shared repository. After changes are pushed, automated build and test stages can validate the application and detect problems early.

Continuous Delivery / Continuous Deployment (CD)

CD takes successfully built and tested code toward release. In Continuous Delivery, the application is kept ready for release and deployment may require approval. In Continuous Deployment, successful changes can be deployed automatically without a manual release step.

Why CI/CD is Useful

- Find defects earlier in the development cycle.
- Reduce repetitive manual deployment work.
- Make releases faster and more predictable.
- Improve consistency between environments.
- Provide traceability from source change to deployed version.

3. AWS Services Used in a Typical Pipeline

AWS Service	Role in CI/CD	Project Status
AWS CodeCommit	Stores and versions source code using Git.	Completed in this project
AWS CodeBuild	Compiles/builds code and runs automated tests.	Next step
AWS CodeDeploy	Automates deployment to supported compute targets.	Next step
AWS CodePipeline	Orchestrates source, build, test, approval, and deployment stages.	Next step
AWS IAM	Controls authentication and authorization to AWS services.	Completed in this project

4. Prerequisites

- An AWS account with permission to use IAM and CodeCommit.
- Git installed on the Windows workstation.
- Git Credential Manager available and configured.
- A terminal such as Git Bash, PowerShell, or Command Prompt.
- Basic HTML editor such as Visual Studio Code, Notepad++, or another text editor.
- Network access to the AWS CodeCommit HTTPS endpoint.

Recommended Local Checks

```
git --version
```

```
git config --global --list
```

The first command confirms Git is installed. The second shows the global Git configuration, including the configured credential helper.

5. Hands-On Implementation



Step 1 - Create an IAM User

Create an IAM user for repository access and grant only the permissions needed for the task. Avoid using the AWS root account for day-to-day Git operations.

Note: Security note: use least privilege. Do not store AWS passwords or access credentials inside the repository.

Step 2 - Create the CodeCommit Repository

Create a repository named, for example, codecommit_demo. The repository becomes the central remote Git location for the project.

Note: After creation, copy the HTTPS clone URL. This URL is used by the local Git client.

Step 3 - Configure Git Credential Manager

Configure Git to use Git Credential Manager for HTTPS authentication.

```
git config --global credential.helper manager
```

Note: This allows Git to securely request and reuse credentials instead of embedding passwords in commands or files.

Step 4 - Clone the Repository

Clone the empty or existing CodeCommit repository to the local workstation.

```
git clone <CodeCommit-Repository-URL>
```

Note: Cloning creates a local working copy and automatically configures the remote named origin.

Step 5 - Enter the Project Directory

Move into the cloned repository before creating or committing files.

```
cd codecommit_demo
```

Step 6 - Create the Application File

Create a simple index.html file to represent the application source code.

Note: The file is only a demonstration page. In a real project this directory would contain application code, configuration, tests, and build files.

Step 7 - Check Repository Status

Review which files are untracked, modified, or staged.

```
git status
```

Note: Run git status frequently. It is one of the safest ways to understand what Git will include in the next commit.

Step 8 - Stage the File

Add the new file or changes to Git staging.

```
git add .
```

Note: For more controlled commits, use git add <filename> instead of staging every change.

Step 9 - Commit the Changes

Create a local Git commit with a meaningful message.

```
git commit -m "Created landing page"
```

Note: A commit records a version of the staged changes in the local Git history.

Step 10 - Push to AWS CodeCommit

Send the local commit to the remote CodeCommit repository.

```
git push origin master
```

Note: If your repository uses main instead of master, push the actual branch name, for example: git push origin main.

Sample Application Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Kaveri - AWS DevOps Project</title>
</head>
<body>
  <h1>Hello, I am Kaveri</h1>
  <h2>My application source code is stored in AWS CodeCommit</h2>
  <p>This is my AWS DevOps practice project.</p>
</body>
</html>
```

6. Understanding the Git Lifecycle

Working Directory	Staging Area	Local Repository	Remote Repository
Edit/create files	git add .	git commit	git push

Simple flow: Edit file → Stage changes → Commit locally → Push to CodeCommit.

7. Verification

Verify Git and Repository State

```
git status
```

```
git log --oneline --decorate -5
```

```
git remote -v
```

- git status should show a clean working tree after the successful commit and push.
- git log should display the Created landing page commit.
- git remote -v should show the CodeCommit URL for origin.
- In the AWS console, the repository should display index.html and the pushed commit.

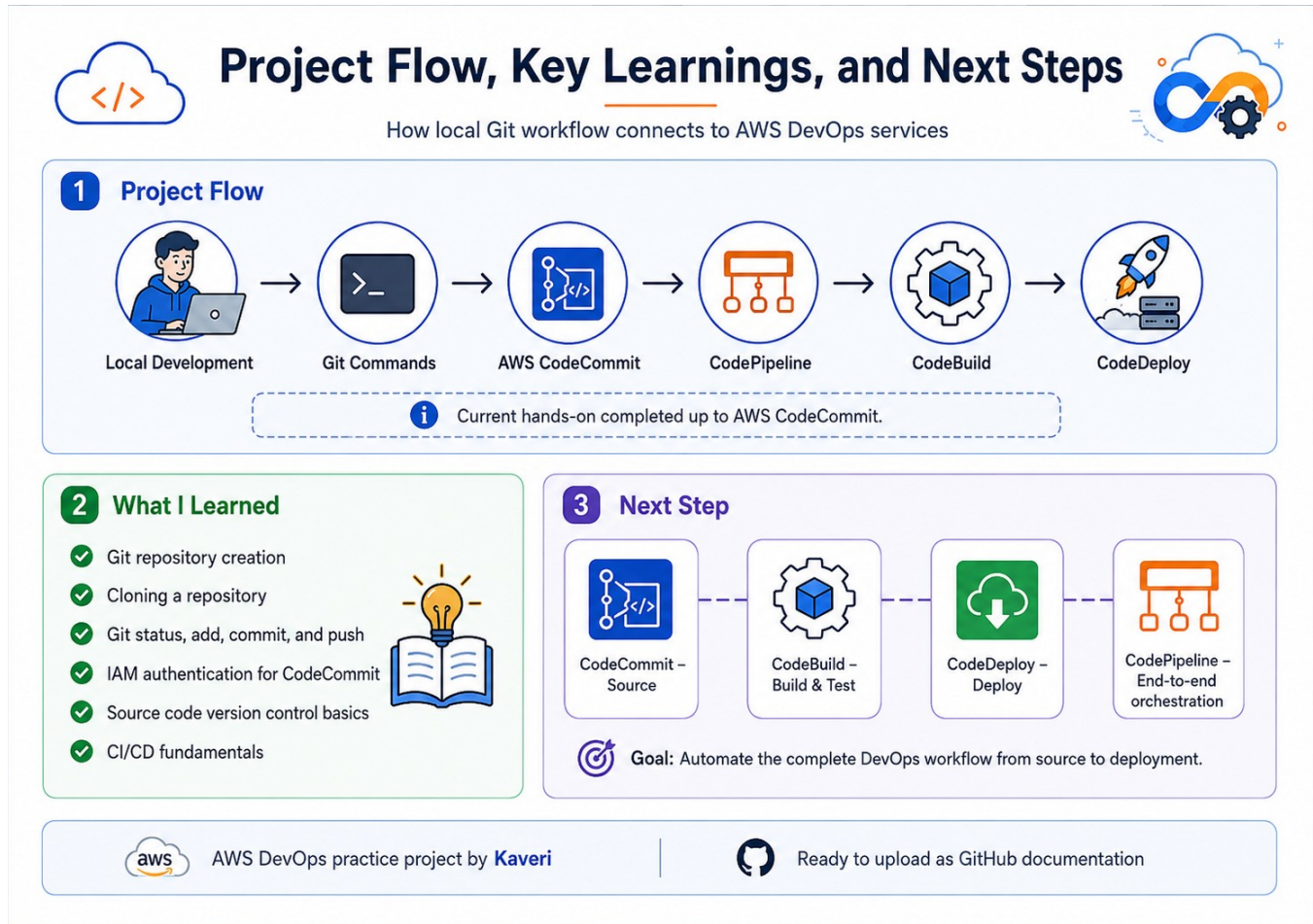
Verify the Application

Open index.html locally in a browser. Confirm that the page loads and displays the project heading and practice-project text. This verifies the application file itself, while the CodeCommit console verifies that version control and remote push were successful.

8. Common Issues and Quick Troubleshooting

Problem	Likely Cause	What to Check
Authentication failed	Incorrect/expired Git credentials or permissions	Verify IAM permissions, HTTPS Git credentials, credential helper, and cached credentials.
Repository not found	Wrong clone URL, region, or repository name	Copy the HTTPS URL again from the correct CodeCommit repository.
Nothing to commit	File is already committed or changes were not saved	Run git status and confirm the file was saved in the repository folder.
Push rejected	Remote branch contains changes not present locally	Run git pull, review/resolve changes, then push again.
Wrong branch name	Repository uses main while command uses master, or vice versa	Run git branch --show-current and push the displayed branch.
index.html opens in Adobe/another app	Windows file association points HTML files to the wrong application	Right-click index.html → Open with → select a web browser, or open the file from the browser.

9. Key Learnings



- How to create and use a Git repository hosted in AWS CodeCommit.
- How Git clone, status, add, commit, and push fit together.
- How IAM permissions and Git credentials protect repository access.
- How local source code is synchronized with a remote repository.
- Why meaningful commits and version history are important for teams.
- How the source repository becomes the first stage of a CI/CD pipeline.

10. Next Phase - Complete the CI/CD Pipeline

The next project phase is to automate the workflow after the source code reaches the repository.

1. CodeBuild

Create a build project and a buildspec.yml file. Use it to validate or build the application automatically.

2. Automated Testing

Add basic validation or application tests so failed code does not continue toward deployment.

3. CodeDeploy

Define the deployment target and deployment instructions. For EC2-based deployments, add the required deployment configuration files and agent setup.

4. CodePipeline

Connect source, build, test, and deploy stages into one end-to-end pipeline. A new source change should automatically trigger the workflow.

Target End-to-End Flow

**Developer → Git Push → CodeCommit → CodePipeline → CodeBuild/Test → CodeDeploy
→ Application**

11. Repository Documentation Checklist

- ☐ Add this document or a README.md explaining the project.
- ☐ Keep index.html in the repository.
- ☐ Create an images/ folder for architecture and workflow diagrams.
- ☐ Never commit passwords, access keys, tokens, or private credentials.
- ☐ Use meaningful commit messages.
- ☐ Add buildspec.yml when CodeBuild is introduced.
- ☐ Add deployment files when CodeDeploy is introduced.
- ☐ Update the architecture diagram as the pipeline grows.

12. Command Reference

Purpose	Command
Configure credential helper	<code>git config --global credential.helper manager</code>
Clone repository	<code>git clone <CodeCommit-Repository-URL></code>
Enter repository	<code>cd codecommit_demo</code>
Check status	<code>git status</code>
Stage changes	<code>git add .</code>
Commit changes	<code>git commit -m "Created landing page"</code>
Check branch	<code>git branch --show-current</code>
Push master	<code>git push origin master</code>
Push main	<code>git push origin main</code>
View recent commits	<code>git log --oneline -5</code>
View remote	<code>git remote -v</code>

Project Status: Source control hands-on completed successfully ✓

Next milestone: Build → Test → Deploy automation